MIT

Training Neural Networks

HIIT

Massachusetts Institute of Technology

Loss Optimization

We want *to* find the *network* weights that achieve the lowest loss

$$W^* = \text{argmin} \frac{1}{n} \sum_{i=1} \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$W^* = \arg\min_W J(W)$$

MIT

HIIT

Massachusetts Institute of

Loss Optimization

We want *to* find the *network* weights that achieve the lowest loss

$$W^* =$$

$$\underset{W}{\text{argmin}} = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}(f(x); W), y()$$

$$W^* = \text{argmin} J(\mathbf{W})$$

MIT

w

Remember:

$$w = \{w^\circ), w(1), \ldots\}$$

Technology

6.S191 Introduction to Deep Learning

introtodeeplearning.com

@MITDeepLearning

1/18/21

$J(wo, W1)$

Loss Optimization

$$W^* = \text{argmin} J(\mathbf{W})$$

w

O

Remember:

Our loss is a function of

the network weights!



$J(wo, W1)$

Loss Optimization

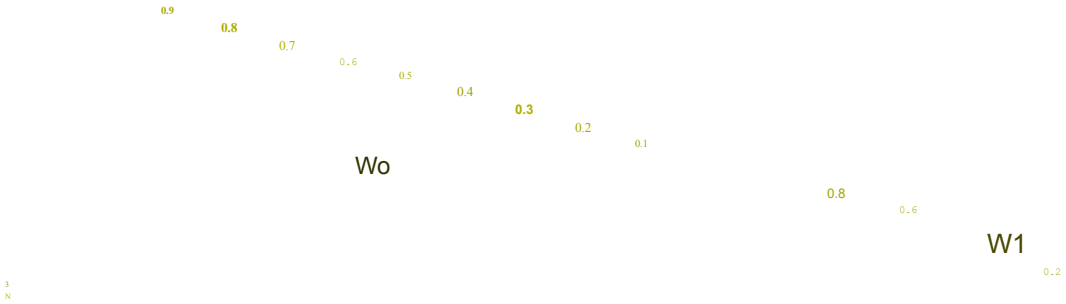
Randomly pick an initial (wo, $W1$)



Loss Optimization

Compute gradient,

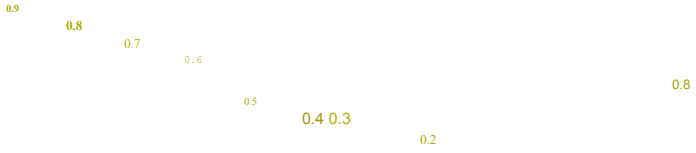
$$\frac{aj(\boldsymbol{w})}{aw}$$



$J(wo, W1)$

Loss Optimization

Take small step in opposite direction of gradient



Wo

W1

$J(wo, W1)$

Gradient Descent

Repeat until convergence

191

0.5 0.4 0.3 Wo

W1

HIIT

Gradient Descent

Algorithm

I. Initialize weights randomly
~N(0,02)

2. Loop until convergence:

Massachusetts
Institute of
Technology

3.

$$\frac{aj(w)}{aw}$$

Compute gradient,

4.

Update weights, $WW-n$ $\frac{aj(w)}{aw}$

5. Return weights

ME

Gradient Descent

1F

Algorithm

1. Initialize weights randomly ~N $(0,\sigma 2)$

2. Loop until convergence:

$$aj(\boldsymbol{w})$$

Compute gradient, aw

3.

4.

Update weights, $W \leftarrow W - \eta \frac{\partial L}{\partial W}$

```
import tensorflow as tf

weights = tf.Variable([tf.random.normal()])

while True: # loop forever
    with tf.GradientTape() as g:
        loss = compute_loss(weights)
    gradient = g.gradient(loss, weights)
    weights = weights - lr * gradient
```

5. Return weights

MIT

Massachusetts Institute of Technology

Gradient Descent

1F

Algorithm

1. Initialize weights randomly ~N $(0,\sigma 2)$
2. Loop until convergence:
   - Compute gradient, $\frac{aj(\boldsymbol{W})}{aw}$
3.
4. Update weights, $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{(w)}{aw}$
5. Return weights

```
import tensorflow as tf
```

```
weights tf.Variable([tf.random.normal()])
                    =

while True: # loop forever
    with tf GradientTape() as g:
        loss = compute_loss (weights)
        gradient g gradient(loss, weights

    weights weights - lr gradient
```

MiT

Massachusetts Institute of Technology

6.S191 **Introduction** to Deep Learning
introtodeeplearning.com

1/18/21

@MITDeepLearning

HIIT

Massachusetts Institu te of Technology

Computing Gradients: Backpropagation

W1

x

W2

9

J(W)

How does a small change *in one weight (ex. w2) affect the final loss* $J(W)$?

HIIT

Massachusetts **Institute of Technology**

Computing Gradients: Backpropagation

W1

*

x

W2

9

$J(W)$

$aj\ (w)$

$aw2$

Let's use the chain rule!

Computing Gradients: Backpropagation

$W1$

$W2$

9

*

x

$J(\mathbf{W})$

Massachusetts
Institute of
Technology

$aj(\mathbf{W})$

MIT

HIIT

$aj\,(\mathbf{w})\;\partial y$

∂y

$aw2$

HIIT

Computing Gradients: Backpropagation



$$\frac{\partial y}{\partial a_j(\mathbf{w})} \cdot \frac{\partial a_j(\mathbf{w})}{\partial dw1}$$

$$\frac{\partial y}{\partial aw1}$$

Apply chain rule!

Apply chain rule!

$J(\mathbf{W})$

Computing Gradients: Backpropagation

x

W1

W2

9

z1

$J(\mathbf{W})$

$aj\ (\boldsymbol{w})$

$aw1$

$aj\ (\boldsymbol{w})\ \partial\acute{y}$

$\partial\check{y}$

az1

$\partial z1$

aw1

Massachusetts Institute of Technology

MI

HIIT

@MITDeepLearning

1/18/21

WIT

Computing Gradients:

Backpropagation

x

Massachusetts
Institute of
Technology

W1

W2

9

z1

*J(***W**)

*aj (**w**)*

*aj (***w**)

∂y̌

az1

aw1

aŷ

∂**z1**

aw1

Repeat this for every weight in the *network using gradients from later layers*

Neural Networks in Practice:
Optimization

MIT

HIIT

Massachusetts **Institute**
of Technology

Training Neural Networks is Difficult

"*Visualizing the loss landscape of neural nets*". *Dec 2017*.

HIIT

Loss Functions Can Be Difficult to Optimize

Massachusetts
Institute of
Technology

Remember:

Optimization through gradient descent

$$W \leftarrow W - \eta$$

MIT

$$aj\ (W)$$

$$aw$$

HIIT

Loss Functions Can Be Difficult to Optimize

Remember:

Optimization through gradient descent

MIT

$$\boldsymbol{W} \leftarrow \mathbf{W} - \eta \frac{aj(\boldsymbol{W})}{aw}$$

How can we set the learning rate?

HIIT

Setting the Learning Rate

Small learning rate converges slowly and *gets stuck* in *false local minima*



T6.S15

HIIT

Massachusetts **Institute of Technology**

Setting the Learning Rate

Large *learning rates overshoot, become unstable* and *diverge*

T6.STS



Initial guess

1/18/21

HIIT

Massachusetts **Institute of Technology**

Setting the Learning **Rate**

Stable learning rates converge smoothly and avoid local minima

T6.S15



6.S191 Introduction to Deep Learning

introtodeeplearning.com

@MITDeepLearning

Initial guess

Massachusetts
Institute of
Technology

How **to** deal **with this?**

Idea I:

Try lots of different learning rates and see what works "just right"

MIT 691

HIIT

HIIT

How **to** deal **with this?**

Idea I:

91

Try lots of different learning rates and see what works "just right"

Idea 2:

Do something smarter!

Design an adaptive learning rate that "adapts" to the landscape

HIIT

Adaptive Learning Rates

Learning rates are no longer fixed

191

Can be made larger or smaller depending on:

- how large gradient is
- how fast learning is happening
- size of particular weights
- etc...

HIIT

Algorithm

- SGD

.

.

Adam

Adadelta

Adagrad

RMSProp

Massachusetts Institute of Technology

Gradient Descent Algorithms

M

TF Implementation

`tf. keras. optimizers. SGD`

`tf.keras. optimizers. Adam`

`tf. keras. optimizers. Adadelta`

`tf.keras optimizers Adagrad`

`tf.keras. optimizers. RMSProp`

Reference

Kiefer & Wolfowitz. "Stochastic Estimation of the Maximum of a Regression Function." 1952.

Kingma et al."Adam: A Method for Stochastic Optimization." 2014.

Zeiler et al. "ADADELTA: An Adaptive Learning Rate Method." 2012.

Duchi et al. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization." 2011.

Additional details: http://ruder.io/optimizing-gradient-descent/

Putting it all together

```
import tensorflow as tf

model = tf.keras. Sequential([
.......])

#pick your favorite optimizer optimizer =
tf.keras optimizer. SGD ()

while True: # loop forever

    # forward pass through the network
    prediction = model(x)

    with tf GradientTape() as tape:
        #compute the loss
        loss = compute_loss (y, prediction)

    #update the weights using the gradient
    grads = tape.gradient(loss, model trainable_variables)
```

Can replace with any Tensorflow optimizer!

```
optimizer.apply_gradients (zip (grads, model
trainable_variables))
```

InstNuts of
Techarologer

MIT

Neural Networks in Practice: Mini-batches

HIIT

Algorithm

Gradient Descent

I. Initialize weights randomly ~N (0,02)

2. Loop until convergence:

3.

4.

$191

Compute gradient, $\frac{aj(\mathbf{w})}{aw}$

Update weights, $\boldsymbol{WW}\text{-}n$

5. Return weights

$\frac{aw}{}$

HIIT

Algorithm

Gradient Descent

I. Initialize weights randomly ~N
$(0,02)$

2. Loop until convergence:

3.

4.

Compute gradient,

$$\frac{aj(w)}{aw}$$

$191

Update weights, ***WW***-n

5. Return weights

$$\frac{aj(\boldsymbol{w})}{a\boldsymbol{w}}$$

Can be very

computationally intensive

to compute!

Algorithm

Stochastic Gradient Descent

I. Initialize weights randomly ~N $(0,02)$

2. Loop until convergence:

3. Pick single data point *i*

4. Compute gradient, $aJi(\mathbf{W})$

5.

G

09 08 07

11

WIT

aw

Update weights, **WW**-n

6. Return weights

Massachusetts **Institute of Technology**

in

weig

hts

aj(**w**)

aw

WIT

Algorithm

Stochastic Gradient Descent

I. Initialize weights randomly ~N $(0,02)$

2. Loop until convergence:

3.

4.

5.

Pick single data point *i*

Compute gradient,

6.$191

$$\frac{aJi(W)}{aw}$$

Update weights, $W \leftarrow w - \eta$

6. Return weights

weig

hts

$$\frac{aj(\mathbf{w})}{aw}$$

Easy to compute but
very noisy (stochastic)!

WIT

Algorithm

Stochastic Gradient Descent

I. Initialize weights randomly ~N (0,02)

2. Loop until convergence:

$191

Jk(W) Σκ=1aw

3. Pick batch of ***B*** data points

4. Compute gradient, $aj(w)$ ΓB aw B

5. Update weights, ***W*** ← ***W***

6. Return weights

wei

ght

S

$- \eta$

$aj(\mathbf{w})$

$\boldsymbol{aw}$

WIT

Algorithm

Stochastic Gradient Descent

I. Initialize weights randomly ~N (0,02)

2. Loop until convergence:

$191

3. Pick batch of ***B*** data points

4. Compute gradient,

$$\frac{aj(w)}{aw} \quad \Gamma B \quad B$$

$$JJk(W)\ \Sigma\kappa=10w \quad aw$$

5. Update weights, $W \leftarrow W - \eta$

6. Return weights

$$\frac{aj(w)}{aw}$$

Massachusetts **Institute of Technology**

Fast to compute and a much better estimate of the true gradient!

Mini-batches while training

More accurate estimation of gradient

Smoother convergence Allows for
larger learning rates

MIT

HIIT

HIIT

Mini-batches while training

191

More accurate **estimation of** gradient

Smoother convergence

Allows for larger learning rates

Mini-batches lead to fast training!

Can parallelize computation + achieve significant speed increases on GPU's

Neural Networks in Practice:

Overfitting

MIT

WIT

The Problem of Overfitting

Underfitting

Model does not have capacity to fully learn the data

Massachusetts **Institute of Technology**

Ideal fit

ثنين

Overfitting

Too complex, extra parameters, does not generalize well

Massachusetts
Institute of
Technology

Regularization

What is it?

Technique that constrains our optimization problem to discourage complex models

MIT 6 191

HIIT

HIIT

Regularization

What *is* **it**?

Technique that constrains our optimization problem to discourage complex models

M191

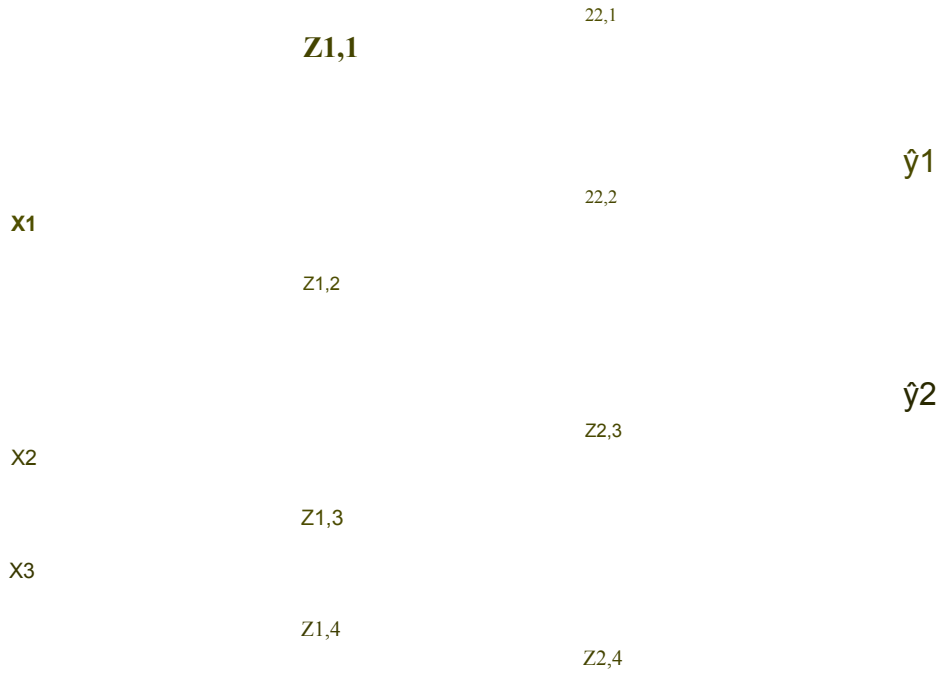
Why do we need it?

Improve generalization of our model on unseen *data*

HIIT

Regularization I: Dropout

During training, randomly set some activations to 0


22,1
Z1,1
ŷ1
22,2
X1
Z1,2
ŷ2
Z2,3
X2
Z1,3
X3
Z1,4
Z2,4

HIIT

Regularization I: Dropout

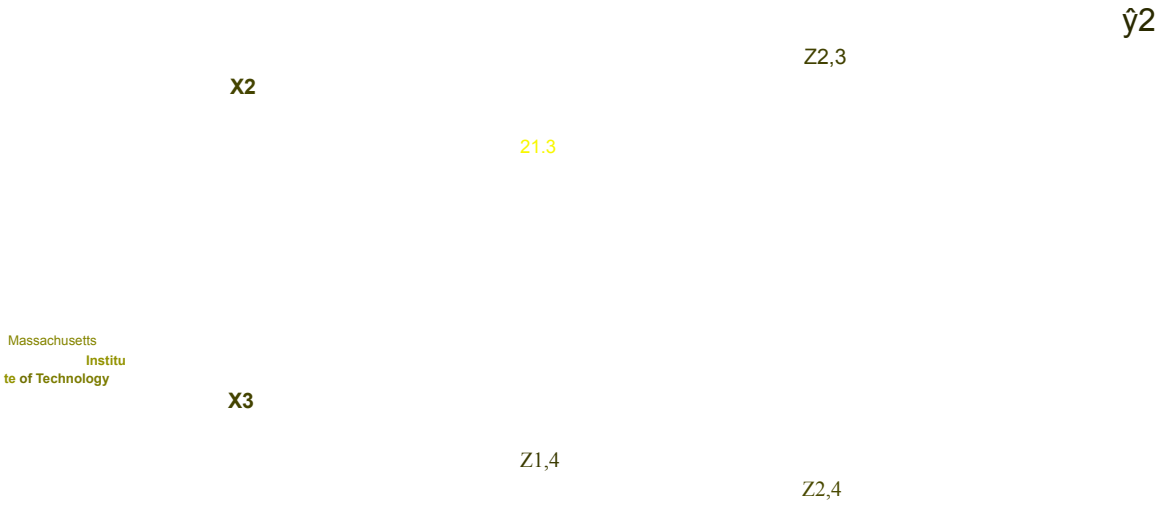
During training, randomly set some activations to 0

- Typically 'drop' 50% of activations in layer

```
tf.keras.layers Dropout (p=0.5)
```

- Forces network to not rely on any I node





6.S191 Introduction to Deep Learning

introtodeeplearning.com

@MITDeepLearning

HIIT

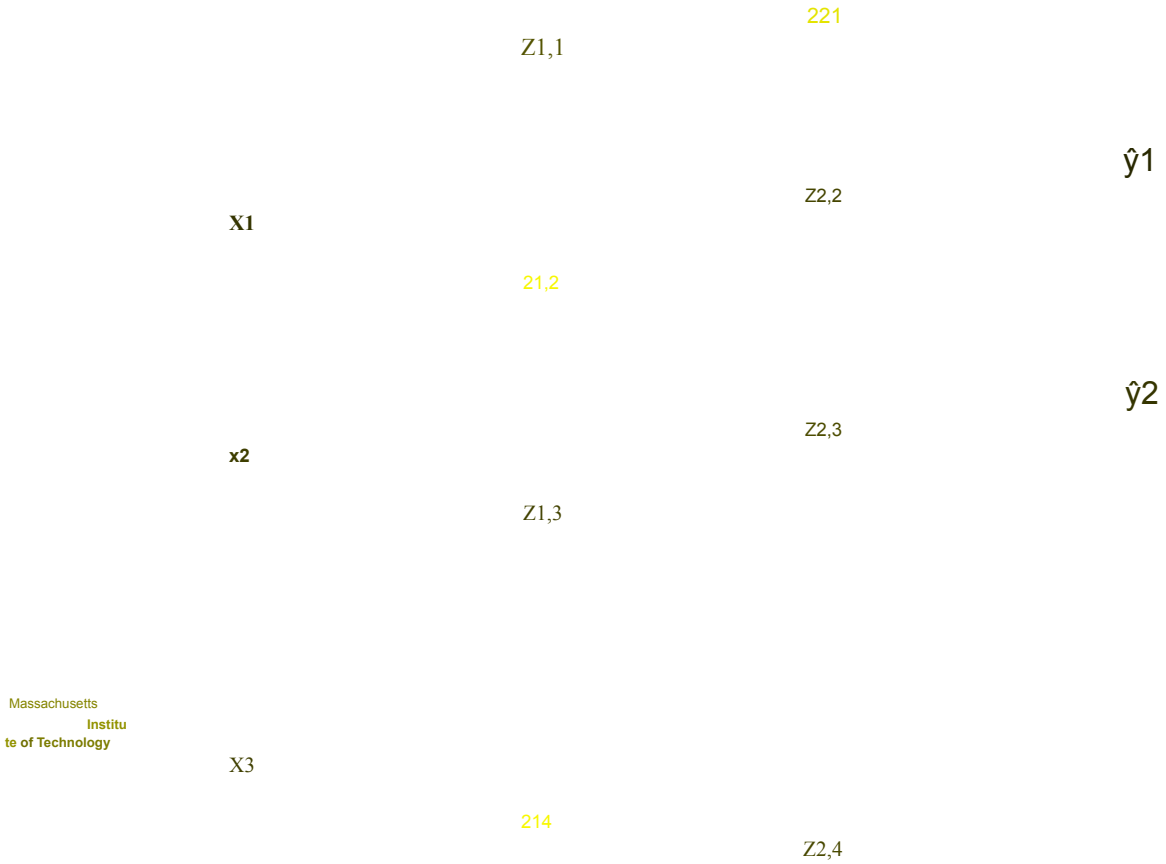
Regularization I: Dropout

During training, randomly set some activations to 0

- Typically 'drop' 50% of activations in layer

```
tf.keras.layers Dropout (p=0.5)
```

- Forces network to not rely on any I node



HIIT

Regularization 2: Early Stopping

Stop training before we have a chance to overfit

Loss

IT 6.S191

Massachusetts Institute of Technology

Training Iterations

6.S191 Introduction to Deep Learning
introtodeeplearning.com
@MITDeepLearning

1/18/21

HIIT

Regularization 2: Early Stopping

Stop training before we have a chance to overfit

Loss

6.S191

Legend

Massachusetts **Institute of Technology**

Training Iterations

6.S191 Introduction to Deep Learning introtodeeplearning.com

Testing

Training

1/18/21

@MITDeepLearning

HIIT

Regularization 2: Early Stopping

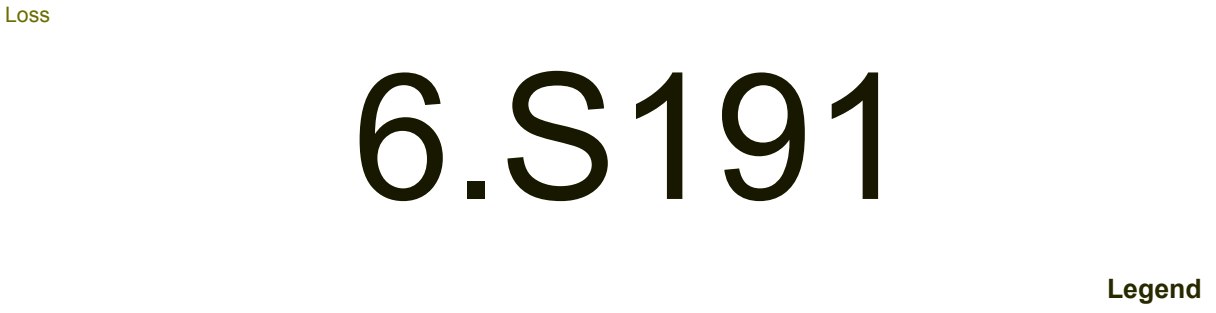
Stop training before we have a chance to overfit



HIIT

Regularization 2: Early Stopping

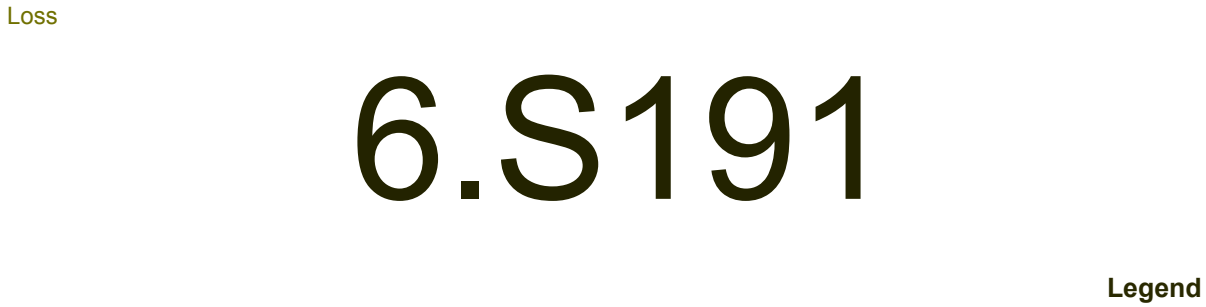
Stop training before we have a chance to overfit



HIIT

Regularization 2: Early Stopping

Stop training before we have a chance to overfit



Testing

Training

HIIT

Regularization 2: Early Stopping

Stop training before we have a chance to overfit

Loss

6.S191

Legend

Testing

Training

HIIT

Regularization 2: Early Stopping

Stop training before we have a chance to overfit

Loss

S191

Stop training
here!

Massachusetts **Institute**
of Technology

Training
Iterations

Legend

Testing

Training

HIIT

Regularization 2: Early Stopping

Stop training before we have a chance to overfit

Under-fitting

91

Loss

Over-fitting

Stop training
here!

Massachusetts **Institute of Technology**

Training
Iterations

Legend

Testing

Training

@MITDeepLearning

HIIT

Core Foundation Review

The Perceptron

Structural building blocks

- Nonlinear activation functions

Neural Networks

Stacking Perceptrons to form neural networks

Optimization through backpropagation

Training in Practice

Adaptive learning

Batching

Regularization

x_1

x_2

x_m

64543

M

X1

X

Xm

X

Zk.1

Zx.2

X

X

92

zk3

Zk.n